AI ASSISTED CODING

ASSIGNMENT-17.1

Name: Hari Priya

H.T.No:2303A51104

Task 1: Perform EDA on Penguin dataset(from seaborn)

• Load and display entire dataset

• Display country wise count

• Visualize bar chart (country wise)

• Visualize bar chart with values count at top

• Visualize pairplot and other plots

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Penguin dataset
penguins_df = sns.load_dataset('penguins')
```

Displaying the entire dataset

```
# Display the first 5 rows of the dataset
display(penguins_df.head())

# Display general information about the dataset
print(penguins_df.info())

# Display descriptive statistics of the dataset
display(penguins_df.describe())
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    object
 1   island             344 non-null    object
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
```

None

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
25%	39.225000	15.600000	190.000000	3550.000000
50%	44.450000	17.300000	197.000000	4050.000000
75%	48.500000	18.700000	213.000000	4750.000000
max	59.600000	21.500000	231.000000	6300.000000

Displaying species-wise count

Since the dataset doesn't have a 'country' column, I'll display the count of observations per 'species', which is a common categorical variable for analysis in this dataset. If you had a different column in mind, please let me know!

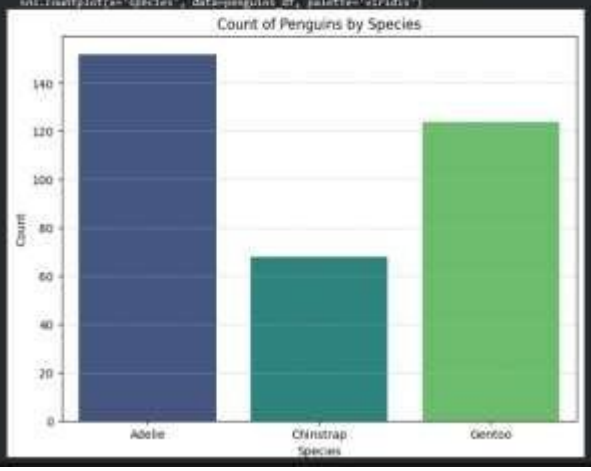
```
# Display the count of each species
species_counts = penguins_df['species'].value_counts()
display(species_counts)
```

species	count
Adelie	152
Gentoo	124
Chinstrap	68

dtype: int64

```
plt.figure(figsize=(8, 6))
sns.countplot(x='species', data=penguins_df, palette='viridis')
plt.title('Count of Penguins by Species')
plt.xlabel('Species')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

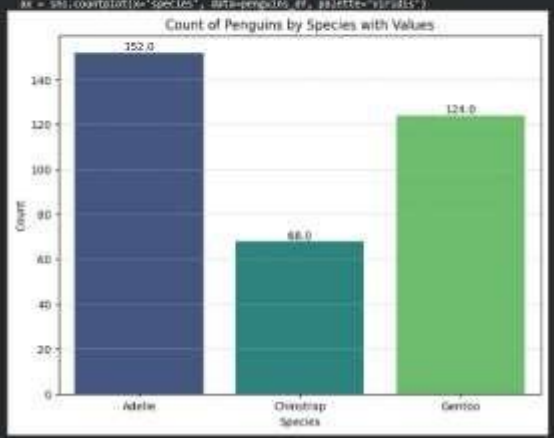
```
/tmp/ipykernel_3779/3461765654.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='species', data=penguins_df, palette='viridis')
```



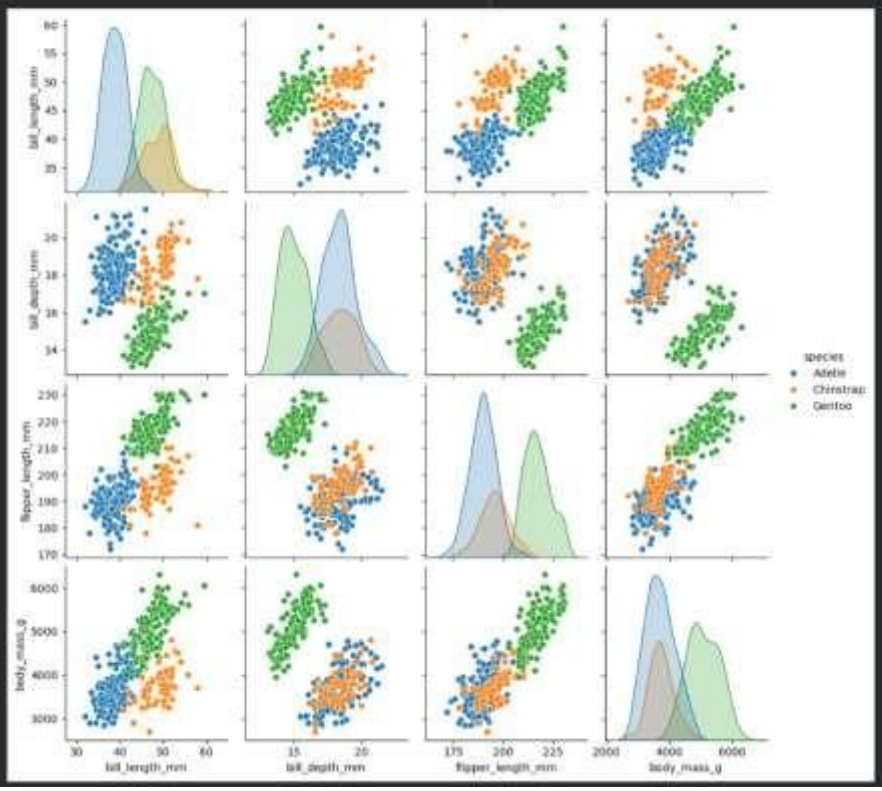
Visualizing bar chart with values count at top

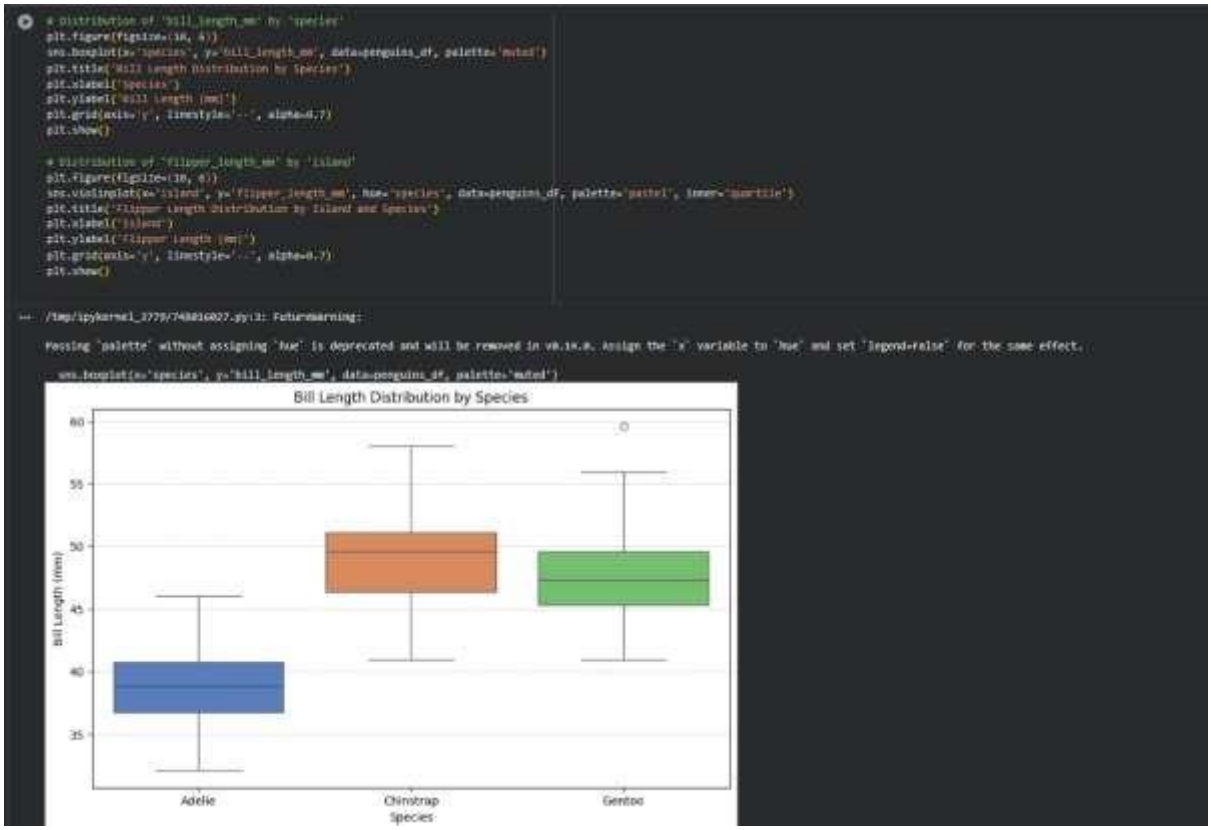
```
plt.figure(figsize=(8, 4))
ax = sns.countplot(x='species', data=penguins_df, palette='viridis')
plt.title('Count of Penguins by Species with Values')
plt.xlabel('Species')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Add the count values on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', fontsize=10, color='black', xytext=(0, 5),
                textcoords='offset points')
plt.show()
```

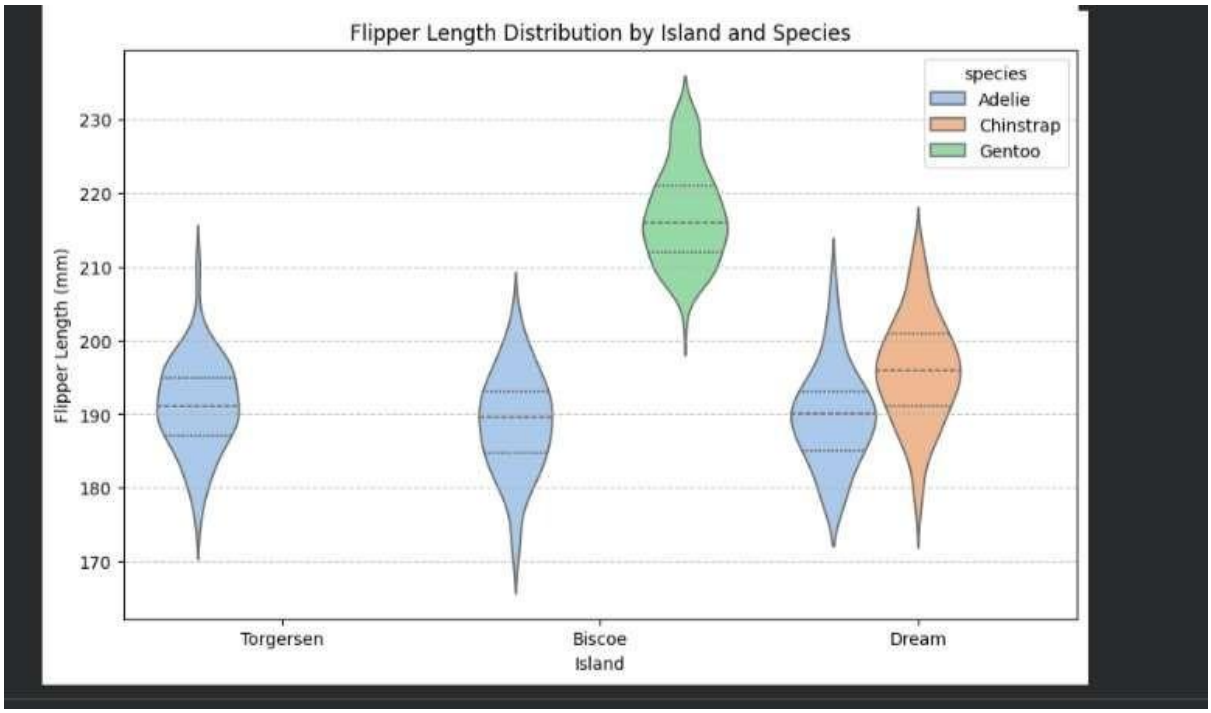
```
/tmp/ipykernel_2779/1104900040.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  ax = sns.countplot(x='species', data=penguins_df, palette='viridis')
```




```
# Pairplot to visualize relationships between numerical variables
sns.pairplot(penguins_df, hue='species', diag_kind='kde')
plt.suptitle('Pairplot of Penguin Dataset by Species', y=1.02) # Adjust title position
plt.show()
```







Task 2: Perform EDA on HealthExp dataset(seaborn)

• Load and display entire dataset

• Display columns and statistical summary(describe)

• Display missing values , and draw bar chart

• Visualize pairplot, regplot and box plot, scatter plot etc

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load the HealthExp dataset
health_exp_df = sns.load_dataset('healthexp')
```

Load and display entire dataset

```
# Display the first 5 rows of the dataset
display(health_exp_df.head())
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

Display columns and statistical summary

```
# Display general information about the dataset
print(health_exp_df.info())

# Display descriptive statistics of the dataset
display(health_exp_df.describe())
```

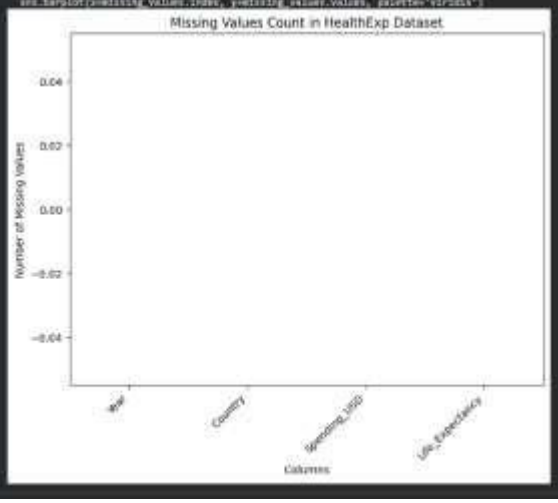
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 274 entries, 0 to 273
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Year             274 non-null    int64
 1   Country          274 non-null    object
 2   Spending_USD     274 non-null    float64
 3   Life_Expectancy  274 non-null    float64
dtypes: float64(2), int64(1), object(1)
memory usage: 8.7+ KB
None
```

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

Display missing values and draw bar chart

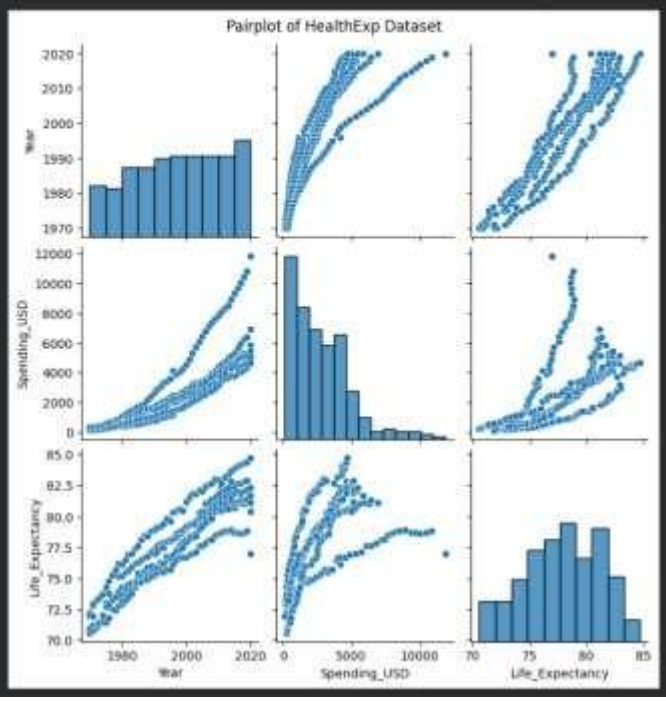
```
# Display missing values count
missing_values = health_exp_df.isnull().sum()
display(missing_values[missing_values > 0])

# Visualize missing values
plt.figure(figsize=(4, 6))
sns.barplot(x=missing_values.index, y=missing_values.values, palette='viridis')
plt.title('Missing Values Count in HealthExp Dataset')
plt.xlabel('Columns')
plt.ylabel('Number of Missing Values')
plt.xticks(rotation=45, ha='right')
plt.show()
```

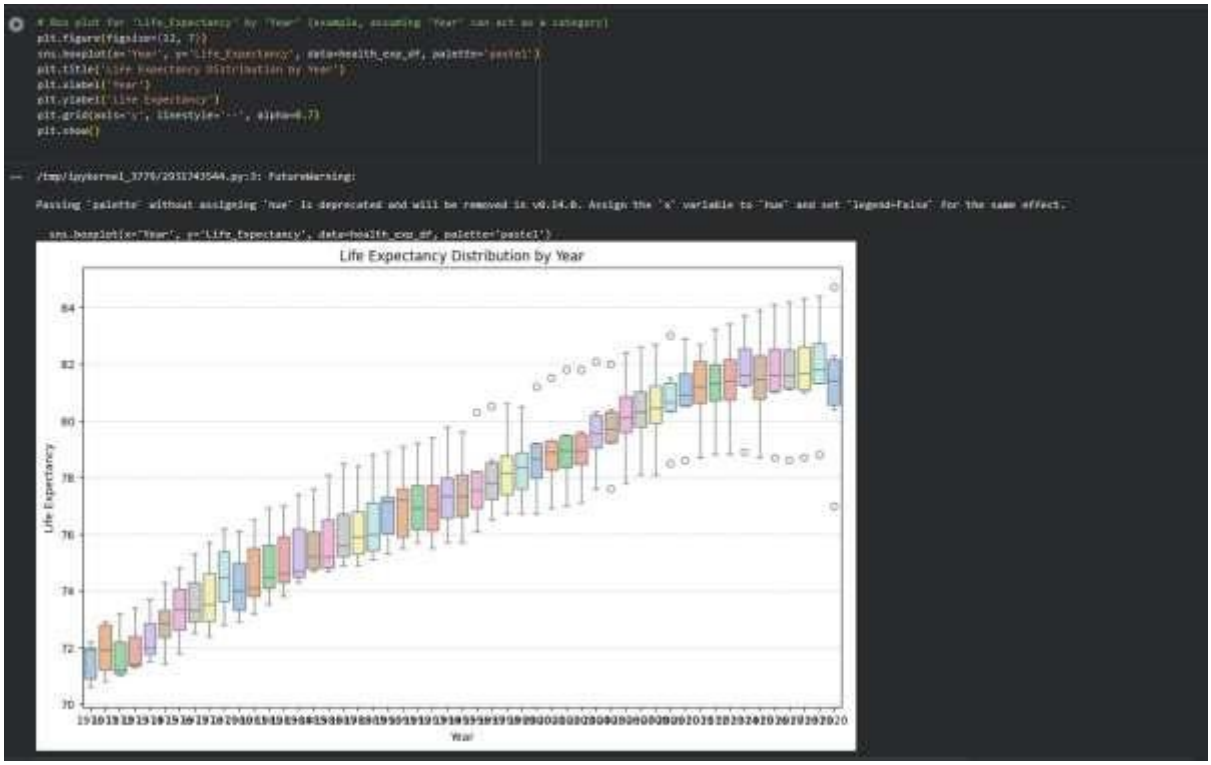
```
0

dtype: int64
/tmp/ipykernel_2776/3899403305.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=missing_values.index, y=missing_values.values, palette='viridis')
```

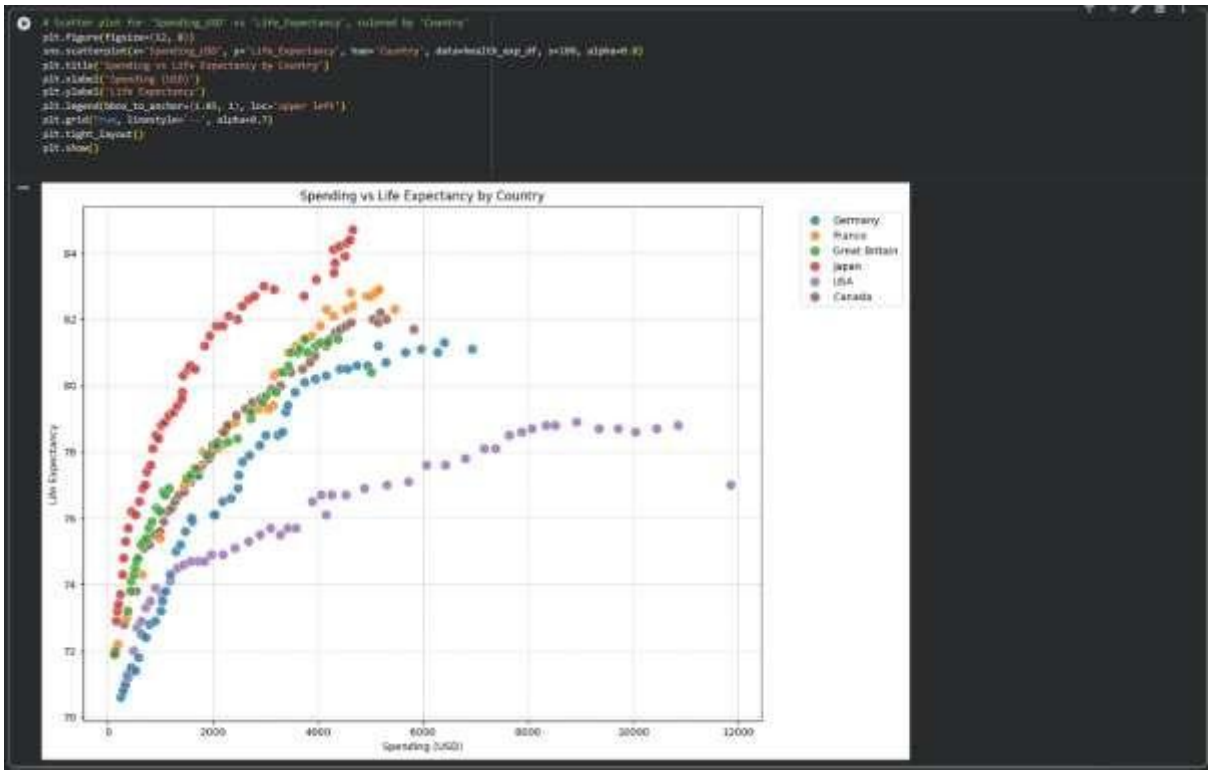


Visualize pairplot, regplot and box plot, scatter plot

```
# Pairplot to visualize relationships between numerical variables
sns.pairplot(health_exp_df)
plt.suptitle('Pairplot of HealthExp Dataset', y=1.02) # Adjust title position
plt.show()
```







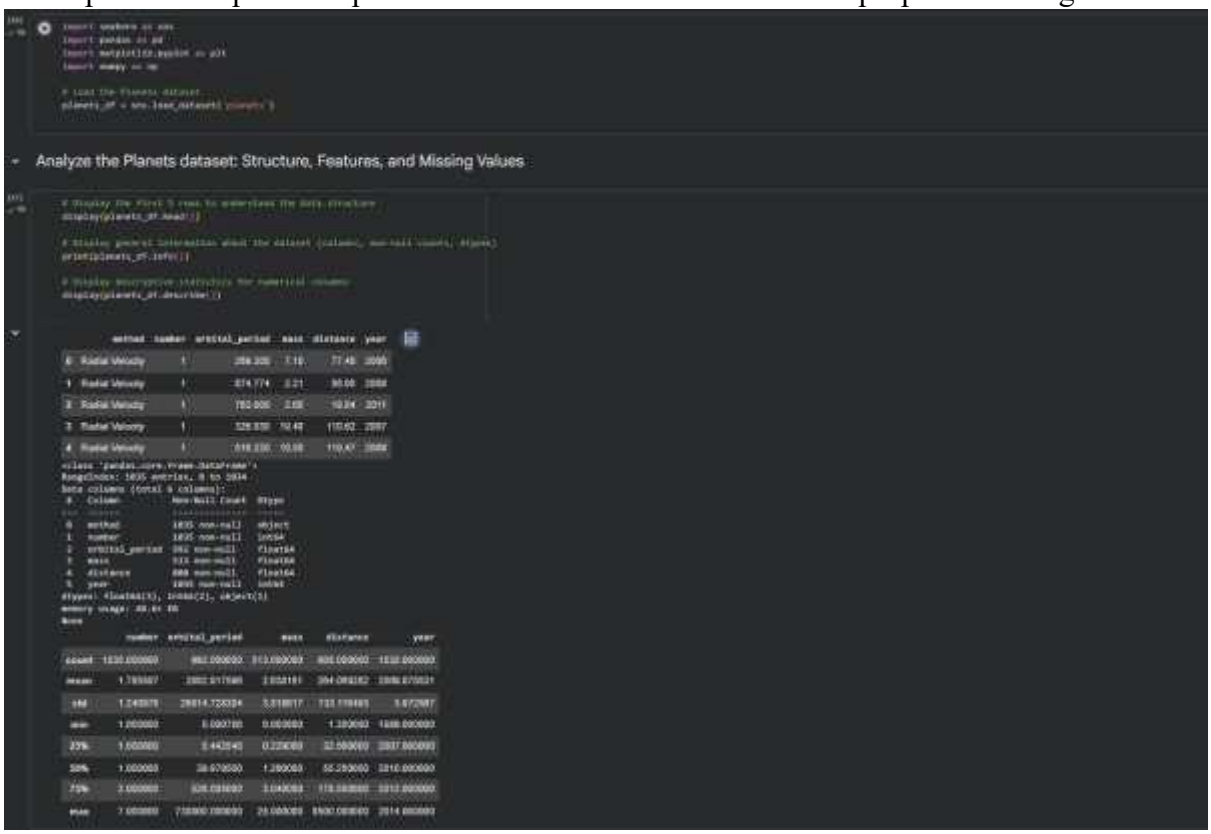
Task 3: Perform following task on given dataset (Train the model)

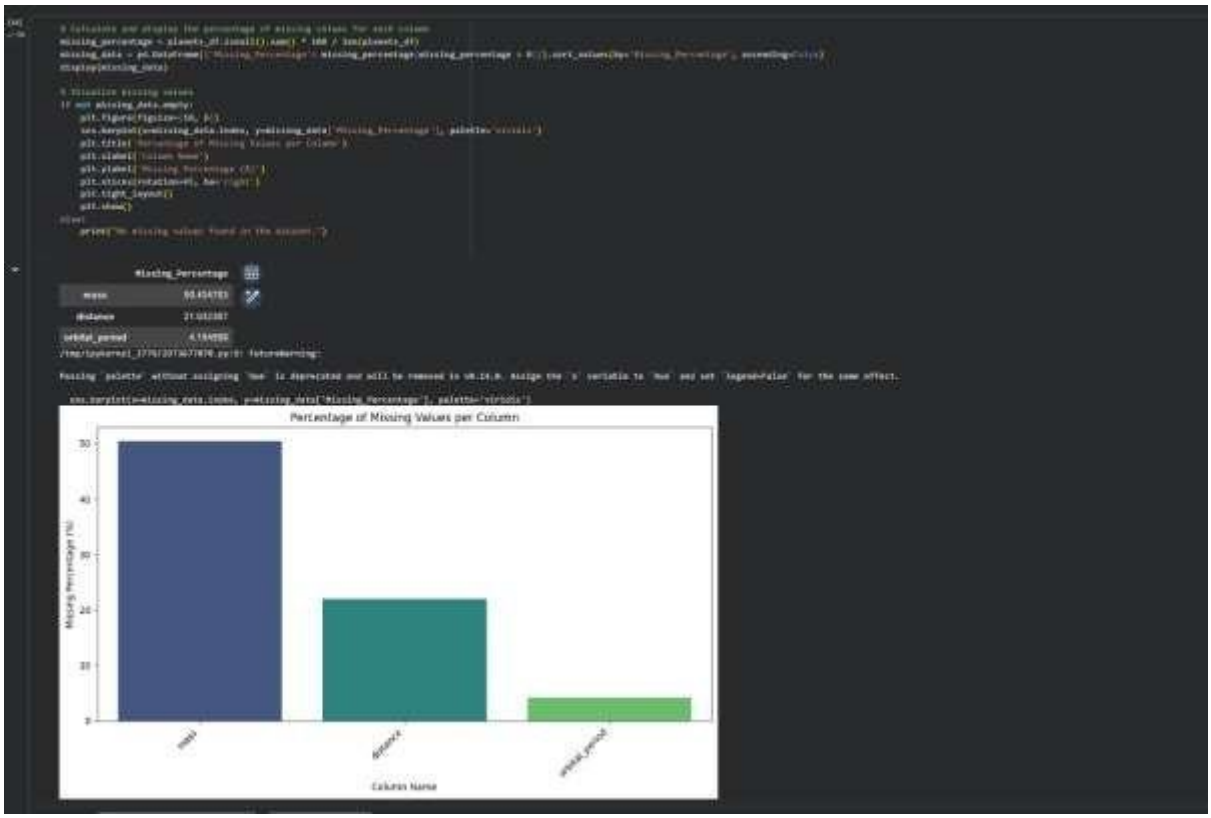
• Analyze the Planets dataset to understand its structure, features, and missing values.

• Perform data cleaning and preprocessing to handle missing or incorrect data.

• Train the dataset using different machine learning models to make predictions.

• Compare and explain the performance of all trained models with proper reasoning.





Perform Data Cleaning and Preprocessing

Based on the missing values analysis, we will address the NaNs. The `orbital_period` column has a significant number of missing values. Since this is a crucial feature, and we aim to predict it, we will drop rows where `orbital_period` is missing. For other numerical columns like `mass` and `distance`, we will impute missing values with the mean. The `method` column is categorical and will be one-hot encoded. The `year` column is important for time-series context, and `number` represents the number of planets found in the system, which can be useful.

We will define `orbital_period` as our target variable (y) and other processed features as our input (X) for a regression task.

```python
# Drop rows where the target variable 'orbital_period' is missing
planets_df_cleaned = planets_df.dropna(subset=['orbital_period']).copy()

# Impute missing values in 'mass' and 'distance' with their respective means
# Using .fillna() on a copy of the DataFrame
planets_df_cleaned['mass'].fillna(planets_df_cleaned['mass'].mean(), inplace=True)
planets_df_cleaned['distance'].fillna(planets_df_cleaned['distance'].mean(), inplace=True)

# Handle categorical features: 'method' column using one-hot encoding
planets_df_processed = pd.get_dummies(planets_df_cleaned, columns=['method'], drop_first=True, dtype=int)

# Display the info of the processed DataFrame to check for remaining NaNs and new columns
print("\nInfo after cleaning and preprocessing:")
print(planets_df_processed.info())

# Display the head of the processed DataFrame
print("\nHead after cleaning and preprocessing:")
display(planets_df_processed.head())
```

```
Info after cleaning and preprocessing:
<class 'pandas.core.frame.DataFrame'>
Index: 992 entries, 0 to 1834
Data columns (total 14 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   number                          992 non-null    int64
 1   orbital_period                  992 non-null    float64
 2   mass                            992 non-null    float64
 3   distance                        992 non-null    float64
 4   year                            992 non-null    int64
 5   method_Eclipse Timing Variations 992 non-null   int64
 6   method_Imaging                  992 non-null    int64
 7   method_Microlensing             992 non-null    int64
 8   method_Orbital Brightness Modulation 992 non-null  int64
 9   method_Pulsar Timing            992 non-null    int64
 10  method_Pulsation Timing Variations 992 non-null  int64
 11  method_Radial Velocity          992 non-null    int64
 12  method_Transit                  992 non-null    int64
 13  method_Transit Timing Variations 992 non-null   int64
dtypes: float64(3), int64(11)
memory usage: 116.2 KB
None

Head after cleaning and preprocessing:
/tmp/ipykernel_5770/2662690.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  planets_df_cleaned['mass'].fillna(planets_df_cleaned['mass'].mean(), inplace=True)
/tmp/ipykernel_5770/2662690.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  planets_df_cleaned['distance'].fillna(planets_df_cleaned['distance'].mean(), inplace=True)
   number  orbital_period  mass  distance  year  method_Eclipse Timing Variations  method_Imaging  method_Microlensing  method_Orbital Brightness Modulation  method_Pulsar Timing  method_Pulsation Timing Variations  method_Radial Velocity  method_Transit  m
0       1          269.300  7.10     77.40  2006                                 0               0                    0                                     0                     0                                   0                       1               0
1       1          874.774  2.21     56.95  2008                                 0               0                    0                                     0                     0                                   0                       1               0
2       1          763.000  2.60     19.84  2011                                 0               0                    0                                     0                     0                                   0                       1               0
3       1          326.030  19.40    110.62  2007                                 0               0                    0                                     0                     0                                   0                       1               0
4       1          516.220  10.50    119.47  2009                                 0               0                    0                                     0                     0                                   0                       1               0
```

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (793, 13)
X_test shape: (199, 13)
y_train shape: (793,)
y_test shape: (199,)
```

Model 1: Linear Regression

```python
# Initialize and train the Linear Regression model
lin_reg_model = LinearRegression()
lin_reg_model.fit(X_train, y_train)

# Make predictions
y_pred_lr = lin_reg_model.predict(X_test)

# Evaluate the model
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print(f"Linear Regression - Mean Squared Error: {mse_lr:.2f}")
print(f"Linear Regression - R-squared: {r2_lr:.2f}")
```

```
Linear Regression - Mean Squared Error: 2768473063.11
Linear Regression - R-squared: 0.12
```

Task 4: Perform following task on given dataset (Train the model)

• Load the MNIST dataset from PyTorch.

• Display basic information and visualize sample input images with their class labels.

• Perform data preprocessing

• Design a EfficientNet model for imageclassification.

• Train the CNN model (10 epochs)

• Evaluate model performance using accuracy and loss metrics.

• Visualize the training history (accuracy and loss curves).

```python
# Import necessary libraries
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Define a transformation to convert images to tensors and normalize them
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Load the MNIST training and test datasets
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

# Create data loaders
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000, shuffle=False)

print(f"Number of training samples: {len(train_dataset)}")
print(f"Number of test samples: {len(test_dataset)}")
print(f"Image shape: {train_dataset[0][0].shape}")
print(f"Number of classes: {len(train_dataset.classes)}")
```

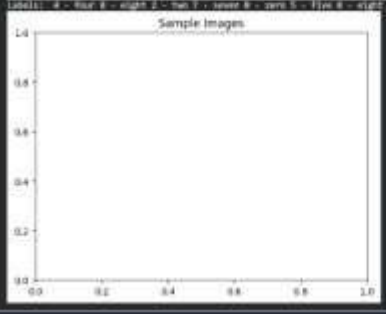
```
Number of training samples: 60000
Number of test samples: 10000
Image shape: torch.Size([1, 28, 28])
Number of classes: 10
```

Visualize sample input images with their class labels

```python
# Function to show an image
def imshow(img):
    img = img / 2 + 0.5  # denormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# Get some random training images
dataiter = iter(train_loader)
images, labels = next(dataiter)

# Show images
plt.figure(figsize=(10, 4))
imshow(torchvision.utils.make_grid(images[:8]))
plt.title('Sample Images')

# Print labels
print('Labels: ', ' '.join('%5s' %train_dataset.classes[labels[j]]for j in range(8)))
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.10780155..2.5107434].
```

```
Labels:  4  8  2  1  0  5  8  6
```



Perform Data Preprocessing

For the MNIST dataset, basic preprocessing usually involves converting images to tensors and normalization. This has already been applied in the `transform` object when loading the dataset. MNIST images are grayscale (1 channel), 28x28 pixels. The `transforms.ToTensor()` converts them to `[C, H, W]` format (1, 28, 28) and scales pixel values to [0, 1]. `transforms.Normalize()` then normalizes these values using the mean and standard deviation of the MNIST dataset.

No further complex preprocessing like resizing or augmentation is typically needed for a standard MNIST classification task with a simple CNN, but it might be considered for more advanced models or higher accuracy goals. Given the task description, the current preprocessing is sufficient.

Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

• Handle missing values in columns (age, income, city).

• Remove duplicate records.

• Standardize text values (e.g., city names in lowercase).

• Encode categorical variables (gender, city).

Sample Code:

import pandas as pd data =

pd.read_csv("shopping_trends.csv")

print(data.head())

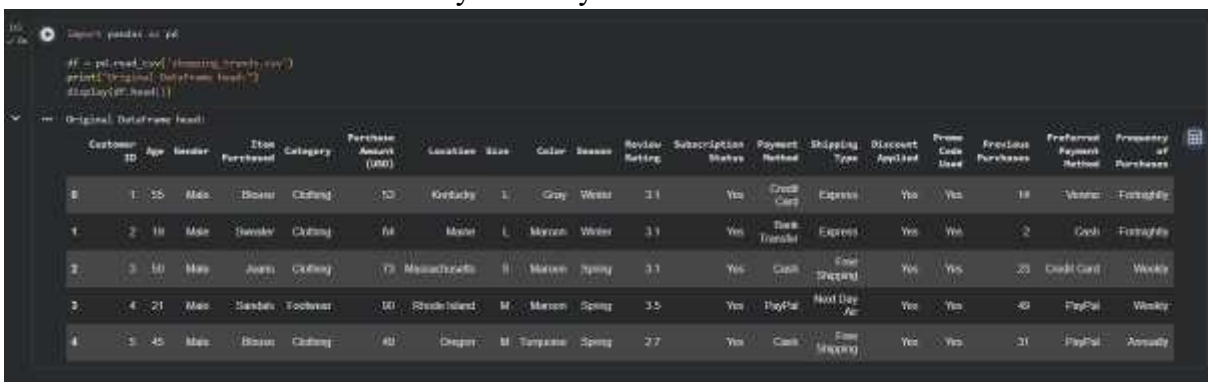
Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/custome r-shopping-trends-

dataset?select=shopping_trends.csv

Expected Output:

• A cleaned Pandas data frame ready for analysis.



```
import pandas as pd

df = pd.read_csv('shopping_trends.csv')
print("Original Dataframe head:")
display(df.head())
```

Original Dataframe head:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

Handling Missing Values

First, let's check for missing values in the specified columns (`age` , `income` , `city`) and then decide on an appropriate strategy. For numerical columns like `age` and `income` , we can fill missing values with the mean or median. For `city` , we might fill with the mode or a placeholder if appropriate, but given it's a categorical variable to be encoded, dropping rows with missing city might be an option depending on the data context. For now, I'll fill numerical columns with their respective means and check the 'City' column.

```
import pandas as pd
df = pd.read_csv('shopping_trends.csv')

print("Missing values before handling:")
display(df[['Age', 'Purchase Amount (USD)', 'Location']].isnull().sum())

# Fill missing 'Age' and 'Purchase Amount (USD)' with their mean
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Purchase Amount (USD)'] = df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean())

# For 'Location' (city), if there are missing values, we can fill with the mode or drop rows.
# Let's check if there are any missing values in 'Location' and fill with mode if they exist.
if df['Location'].isnull().any():
    mode_city = df['Location'].mode()[0]
    df['Location'] = df['Location'].fillna(mode_city)

print("\nMissing values after handling:")
display(df[['Age', 'Purchase Amount (USD)', 'Location']].isnull().sum())
```

```
Missing values before handling:
                          0
Age                       0
Purchase Amount (USD)     0
Location                  0
dtype: int64

Missing values after handling:
                          0
Age                       0
Purchase Amount (USD)     0
Location                  0
dtype: int64
```

```
# Encode 'Gender' using map
df['Gender_Encoded'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Encode 'Location' using One-Hot Encoding
df = pd.get_dummies(df, columns=['Location'], prefix='Location')

print("Dataframe head after encoding 'Gender' and 'Location':")
display(df.head())
```

Dataframe head after encoding 'Gender' and 'Location':

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Size	Color	Season	Review Rating	...	Location_south dakota	Location_tennessee	Location_texas	Location_utah	Location_vermont	Location_virginia
0	1	55	Male	Blouse	Clothing	53	L	Gray	Winter	3.1	...	False	False	False	False	False	False
1	2	19	Male	Sweater	Clothing	64	L	Maroon	Winter	3.1	...	False	False	False	False	False	False
2	3	50	Male	Jeans	Clothing	73	S	Maroon	Spring	3.1	...	False	False	False	False	False	False
3	4	21	Male	Sandals	Footwear	90	M	Maroon	Spring	3.5	...	False	False	False	False	False	False
4	5	45	Male	Blouse	Clothing	49	M	Turquoise	Spring	2.7	...	False	False	False	False	False	False

5 rows × 68 columns

The data has now been cleaned according to your specifications. You can see the new `Gender_Encoded` column and the one-hot encoded `Location` columns in the DataFrame.

Task 6 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

• Convert date columns into datetime format.

• Create new features (month, year, day of week).

• Normalize sales amount column.

• Detect and handle outliers in dataset.

Sample Code:

import pandas as pd data =

pd.read_csv("sales_data.csv")

print(data.head())

Dataset link: https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

• A transformed dataset with time-based features and normalized values.

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load the sales data
sales_df = pd.read_csv('sales_data.csv')
print("Original sales_data DataFrame head:")
display(sales_df.head())

print("\nDataFrame Info:")
sales_df.info()
```

```
Original sales_data DataFrame head:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	1052	2023-02-03	Bob	North	5053.87	18	Furniture	152.75	267.22	Returning	0.08	Cash	Online	North-Bob
1	1060	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4208.44	Returning	0.11	Cash	Retail	West-Bob
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Product_ID          1000 non-null   int64
 1   Sale_Date           1000 non-null   object
 2   Sales_Rep           1000 non-null   object
```

1. Convert Date Columns to Datetime Format & Create New Features

Based on the `df.info()` output, it seems `OrderDate` is a good candidate for a date column. I will convert it to datetime objects and then extract 'year', 'month', and 'day of week' as new features.

```python
# Convert 'Sale_Date' to datetime
sales_df['Sale_Date'] = pd.to_datetime(sales_df['Sale_Date'])

# Create new features
sales_df['OrderYear'] = sales_df['Sale_Date'].dt.year
sales_df['OrderMonth'] = sales_df['Sale_Date'].dt.month
sales_df['OrderDayOfWeek'] = sales_df['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6

print("DataFrame head with new date features:")
display(sales_df.head())
```

```
DataFrame head with new date features:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Order
0	1052	2023-02-03	Bob	North	5053.87	18	Furniture	152.75	267.22	Returning	0.08	Cash	Online	North-Bob	
1	1060	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4208.44	Returning	0.11	Cash	Retail	West-Bob	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	

2. Normalize Sales Amount Column

I'll assume `SaleAmount` is the column to normalize. I will use Min-Max scaling to transform the values to a range between 0 and 1.

```python
# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Normalize 'Sales_Amount'
sales_df['SaleAmount_Normalized'] = scaler.fit_transform(sales_df[['Sales_Amount']])

print("DataFrame head with normalized 'SaleAmount_Normalized':")
display(sales_df.head())
```

```
DataFrame head with normalized 'SaleAmount_Normalized':
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	OrderYear
	1052	2023-02-03	Bob	North	5053.87	18	Furniture	152.75	267.22	Returning	0.08	Cash	Online	North-Bob	2023
	1060	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4208.44	Returning	0.11	Cash	Retail	West-Bob	2023
	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	2023
	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	2023
	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	2023

3. Detect and Handle Outliers

I will detect outliers in the `SaleAmount` column using the Interquartile Range (IQR) method. Outliers will be capped at the upper and lower bounds to mitigate their impact.

```python
# Calculate IQR for 'Sales_Amount'
Q1 = sales_df['Sales_Amount'].quantile(0.25)
Q3 = sales_df['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q1 + 1.5 * IQR

# Detect outliers
outliers_count = sales_df[(sales_df['Sales_Amount'] < lower_bound) | (sales_df['Sales_Amount'] > upper_bound)].shape[0]
print(f"Number of outliers detected in 'Sales_Amount': {outliers_count}")

# Handle outliers by capping
sales_df['SaleAmount_Capped'] = sales_df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)

print("\nDataframe head with 'SaleAmount_Capped' (outliers handled):")
display(sales_df.head())

# Verify if capping worked by checking descriptive statistics
print("\nDescriptive statistics for original and capped 'Sales_Amount':")
display(sales_df[['Sales_Amount', 'SaleAmount_Capped']].describe())
```

```
Number of outliers detected in 'Sales_Amount': 0

Dataframe head with 'SaleAmount_Capped' (outliers handled):
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	OrderYear	OrderMonth	OrderDayOfWeek	SaleAmount
0	1012	2023-02-01	Bob	North	5263.02	16	Furniture	152.75	267.21	Returning	0.03	Cash	Online	North-Bob	2023	2	4	
1	1030	2023-04-21	Bob	West	4304.03	12	Furniture	3818.30	4209.44	Returning	0.11	Cash	Retail	West-Bob	2023	4	4	
2	1016	2023-09-21	David	South	4819.73	30	Food	201.56	371.48	Returning	0.20	Bank Transfer	Retail	South-David	2023	9	3	
3	1072	2023-09-24	Bob	South	2957.94	39	Clothing	4138.93	4407.75	New	0.03	Credit Card	Retail	South-Bob	2023	9	3	
4	1001	2023-03-24	Charlie	East	5758.28	15	Electronics	697.37	992.73	New	0.00	Credit Card	Online	East-Charlie	2023	3	4	

```
Descriptive statistics for original and capped 'Sales_Amount':
```

	Sales_Amount	SaleAmount_Capped
count	[illegible]	[illegible]
mean	[illegible]	[illegible]
std	[illegible]	[illegible]
min	[illegible]	[illegible]
25%	[illegible]	[illegible]
50%	[illegible]	[illegible]
75%	[illegible]	[illegible]
max	[illegible]	[illegible]

Task 7 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

• Handle missing values in blood_pressure, cholesterol.

• Encode categorical features

• Scale numerical features (min-max or standard scaling).

• Split dataset into training and testing sets.

Dataset link :

https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset

Expected Output:

• A preprocessed dataset ready for machine learning models.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
import numpy as np

# Load the healthcare dataset.
# Assuming the dataset is named 'healthcare_data.csv'. Please change if different.
try:
    df_health = pd.read_csv('healthcare_data.csv')
    print("Successfully loaded healthcare_data.csv")
except FileNotFoundError:
    print("healthcare_data.csv not found. Please upload the file or provide the correct path/name.")
    # Creating a dummy DataFrame for demonstration if file not found
    data = {
        'patient_id': range(1, 101),
        'age': np.random.randint(18, 80, 100),
        'gender': np.random.choice(['Male', 'Female'], 100),
        'blood_pressure': np.random.normal(90, 140, 100) + np.random.choice([np.nan, 0], 100, p=[0.1, 0.9]),
        'cholesterol': np.random.normal(150, 250, 100) + np.random.choice([np.nan, 0], 100, p=[0.01, 0.99]),
        'heart_rate': np.random.randint(50, 100, 100),
        'smoking_status': np.random.choice(['Yes', 'No'], 100),
        'medication': np.random.choice(['Drug', 'Drug', 'Drug'], 100)
    }
    df_health = pd.DataFrame(data)
    print("Created a dummy DataFrame for demonstration.")

print("healthcare DataFrame head:")
display(df_health.head())

print("\nDataFrame Info:")
df_health.info()
```

```
healthcare_data.csv not found. Please upload the file or provide the correct path/name.
Created a dummy DataFrame for demonstration.

healthcare DataFrame head:
```

	patient_id	age	gender	blood_pressure	cholesterol	heart_rate	smoking_status	medication
0	1	56	Female	125.0	220.0	76	Yes	Drug
1	2	51	Female	120.0	150.0	94	Yes	Drug
2	3	32	Female	NaN	153.0	77	Yes	Drug
3	4	48	Female	80.0	154.0	93	Yes	Drug
4	5	71	Male	135.0	221.0	87	No	Drug

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   patient_id      100 non-null    int64
 1   age             100 non-null    int64
 2   gender          100 non-null    object
 3   blood_pressure  90 non-null     float64
 4   cholesterol     99 non-null     float64
 5   heart_rate      100 non-null    int64
 6   smoking_status  100 non-null    object
 7   medication      100 non-null    object
dtypes: float64(2), int64(3), object(3)
memory usage: 6.4+ KB
```

1. Handle Missing Values

I will handle missing values in `blood_pressure` and `cholesterol` columns by filling them with the mean of their respective columns. This is a common strategy for numerical features.

```python
print("Missing values before handling:")
display(df_health[['blood_pressure', 'cholesterol']].isnull().sum())

# Fill missing blood_pressure with its mean
df_health['blood_pressure'] = df_health['blood_pressure'].fillna(df_health['blood_pressure'].mean())

# Fill missing cholesterol with its mean
df_health['cholesterol'] = df_health['cholesterol'].fillna(df_health['cholesterol'].mean())

print("\nMissing values after handling:")
display(df_health[['blood_pressure', 'cholesterol']].isnull().sum())
```

```
Missing values before handling:
```

	0
blood_pressure	10
cholesterol	1

```
dtype: int64

Missing values after handling:
```

	0
blood_pressure	0
cholesterol	0

```
dtype: int64
```

2. Encode Categorical Features

Next, I'll identify and encode categorical features. For nominal categorical features, One-Hot Encoding is a suitable method. I'll automatically detect object-type columns as categorical.

```
# Identify categorical features
categorical_features = df_health.select_dtypes(include='object').columns
print(f"Categorical features identified: {list(categorical_features)}")

# Apply One-Hot Encoding
df_health_encoded = pd.get_dummies(df_health, columns=categorical_features, drop_first=True)

print("\nDataframe head after encoding categorical features:")
display(df_health_encoded.head())
```

```
Categorical features identified: ['gender', 'smoking_status', 'medication']

Dataframe head after encoding categorical features:
```

	patient_id	age	blood_pressure	cholesterol	heart_rate	gender_Male	smoking_status_Yes	medication_DrugB	medication_DrugC
0	1	59	129.000000	298.0	75	False	True	True	False
1	2	51	138.000000	165.0	94	False	True	True	False
2	3	32	129.444444	153.0	77	False	True	False	False
3	4	48	95.000000	154.0	93	False	True	True	False
4	5	51	101.000000	223.0	87	True	False	True	False

3. Scale Numerical Features

I will scale the numerical features using `MinMaxScaler` to bring them into a common range (0 to 1). This is important for many machine learning algorithms. I will exclude 'patient_id' from scaling if it exists, as it's typically an identifier and not a feature.

```
# Identify numerical features for scaling (excluding patient_id and already encoded/handled columns)
numerical_features = df_health_encoded.select_dtypes(include=['int64', 'float64']).columns.tolist()
if 'patient_id' in numerical_features:
    numerical_features.remove('patient_id')

print(f"Numerical features to scale: {numerical_features}")

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Apply scaling
df_health_encoded[numerical_features] = scaler.fit_transform(df_health_encoded[numerical_features])

print("\nDataframe head after scaling numerical features:")
display(df_health_encoded.head())
```

```
Numerical features to scale: ['age', 'blood_pressure', 'cholesterol', 'heart_rate']

Dataframe head after scaling numerical features:
```

	patient_id	age	blood_pressure	cholesterol	heart_rate	gender_Male	smoking_status_Yes	medication_DrugB	medication_DrugC
0	1	0.661017	0.443182	0.693548	0.364615	False	True	True	False
1	2	0.525424	0.545455	0.346774	0.871795	False	True	True	False
2	3	0.203390	0.448232	0.250000	0.435897	False	True	False	False
3	4	0.474576	0.056818	0.258065	0.840154	False	True	True	False
4	5	0.525424	0.125000	0.814516	0.692308	True	False	True	False

4. Split Dataset into Training and Testing Sets

Finally, I will split the preprocessed dataset into training and testing sets. You'll need to define your target variable (y) and features (X).
For this example, I'll assume `heart_rate` is the target variable, but you should adjust this based on your specific task.

```
# Define features (X) and target (y)
# Assuming 'heart_rate' is the target variable. Adjust if your target is different.
if 'heart_rate' in df_health_encoded.columns:
    X = df_health_encoded.drop(columns=['heart_rate', 'patient_id'], errors='ignore') # Drop patient_id as well
    y = df_health_encoded['heart_rate']
    print("Target variable set to 'heart_rate'.")
else:
    print("Could not find 'heart_rate' column. Please define your target variable (y).")
    X = df_health_encoded.drop(columns=['patient_id'], errors='ignore') # Drop patient_id if no target
    y = pd.Series(np.zeros(len(X))) # Dummy target if not found

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("\nShape of X_train: {X_train.shape}")
print("Shape of X_test: {X_test.shape}")
print("Shape of y_train: {y_train.shape}")
print("Shape of y_test: {y_test.shape}")

print("\nData preprocessing complete. The dataset is now split into training and testing sets.")
```

```
Target variable set to 'heart_rate'.

Shape of X_train: (80, 7)
Shape of X_test: (20, 7)
Shape of y_train: (80,)
Shape of y_test: (20,)

Data preprocessing complete. The dataset is now split into training and testing sets.
```

Task 8 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

• Handle missing values

• Detect and remove salary outliers.

• Standardize department names (lowercase).

• Apply label encoding to job location, department.

Dataset link:

https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output:

• A structured dataset ready for HR analytics.

```
[26]
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import numpy as np

# Load the employee salary dataset
# Assuming the dataset is named 'employee_salary.csv'. Please change if different.
try:
    df_salary = pd.read_csv('employee_salary.csv')
    print("Successfully loaded employee_salary.csv")
except FileNotFoundError:
    print("employee_salary.csv not found. Creating a dummy DataFrame for demonstration.")
    # Creating a dummy DataFrame for demonstration if file not found
    data = {
        'Employee_ID': range(1, 101),
        'Age': np.random.randint(22, 60, 100),
        'Department': np.random.choice(['Sales', 'Marketing', 'IT', 'HR', 'Finance'], 100),
        'Job_Location': np.random.choice(['New York', 'London', 'Paris', 'Tokyo'], 100),
        'Years_Experience': np.random.randint(0, 30, 100),
        'Salary': np.random.randint(30000, 120000, 100).astype(float) # Ensure salary is float to allow NaN
    }
    # Introduce some missing values and outliers for demonstration
    data['Salary'][np.random.choice(100, 5)] = np.nan # 5 missing salaries
    data['Salary'][np.random.choice(100, 2)] = 500000 # 2 salary outliers
    data['Department'][np.random.choice(100, 3)] = np.nan # 3 missing departments
    data['Job_Location'][np.random.choice(100, 2)] = np.nan # 2 missing job locations
    df_salary = pd.DataFrame(data)

print("\nEmployee Salary DataFrame Head:")
display(df_salary.head())

print("\nDataFrame Info:")
df_salary.info()

print("\nMissing values before handling:")
display(df_salary.isnull().sum())
```

```
employee_salary.csv not found. Creating a dummy DataFrame for demonstration.

Employee Salary DataFrame Head:
```

	Employee_ID	Age	Department	Job_Location	Years_Experience	Salary
0	1	40	Sales	London	17	80587.0
1	2	54	IT	nan	9	31826.0
2	3	31	IT	Tokyo	17	76778.0
3	4	40	Sales	New York	3	90560.0
4	5	39	IT	London	12	30918.0

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Employee_ID       100 non-null    int64
 1   Age               100 non-null    int64
 2   Department        100 non-null    object
 3   Job_Location      100 non-null    object
 4   Years_Experience  100 non-null    int64
 5   Salary            96 non-null     float64
dtypes: float64(1), int64(3), object(2)
memory usage: 4.8+ KB

Missing values before handling:
```

	0
Employee_ID	0
Age	0
Department	0
Job_Location	0
Years_Experience	0
Salary	4

```
dtype: int64
```

1. Handle Missing Values

I will handle missing values for numerical columns (like 'Salary' and 'Years_Experience') by filling them with the mean. For categorical columns ('Department', 'Job_Location'), I will fill missing values with the mode.

```
# Fill missing numerical values with the mean
for col in ['Salary', 'Years_Experience']:
    if df_salary[col].isnull().any():
        df_salary[col] = df_salary[col].fillna(df_salary[col].mean())

# Fill missing categorical values with the mode
for col in ['Department', 'Job_Location']:
    if df_salary[col].isnull().any():
        df_salary[col] = df_salary[col].fillna(df_salary[col].mode()[0])

print("Missing values after handling:")
display(df_salary.isnull().sum())
```

```
Missing values after handling:
```

	0
Employee_ID	0
Age	0
Department	0
Job_Location	0
Years_Experience	0
Salary	0

```
dtype: int64
```

2. Detect and Remove Salary Outliers

I will use the Interquartile Range (IQR) method to detect outliers in the 'Salary' column. Rows identified as outliers will be removed from the dataset.

```
Q1 = df_salary['Salary'].quantile(0.25)
Q3 = df_salary['Salary'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers_before_removal = df_salary[(df_salary['Salary'] < lower_bound) | (df_salary['Salary'] > upper_bound)]
print(f"Number of salary outliers detected: {len(outliers_before_removal)}")

# Remove outliers
df_salary_cleaned = df_salary[~((df_salary['Salary'] < lower_bound) | (df_salary['Salary'] > upper_bound))].copy()

print(f"Shape before outlier removal: {df_salary.shape}")
print(f"Shape after outlier removal: {df_salary_cleaned.shape}")

print("\nDataFrame head after outlier removal:")
display(df_salary_cleaned.head())
```

```
Number of salary outliers detected: 2
Shape before outlier removal: (100, 6)
Shape after outlier removal: (98, 6)

DataFrame head after outlier removal:
```

	Employee_ID	Age	Department	Job_Location	Years_Experience	Salary
0	1	40	Sales	London	17	80507.0
1	2	54	IT	nan	9	31826.0
2	3	31	IT	Tokyo	17	70778.0
3	4	40	Sales	New York	3	90560.0
4	5	30	IT	London	12	30910.0

3. Standardize Department Names (lowercase)

I will convert all department names in the 'Department' column to lowercase to ensure consistency.

```
print("Department names before standardization (sample):")
display(df_salary_cleaned['Department'].value_counts().head())

df_salary_cleaned['Department'] = df_salary_cleaned['Department'].str.lower()

print("\nDepartment names after standardization (sample):")
display(df_salary_cleaned['Department'].value_counts().head())
```

```
Department names before standardization (sample):
```

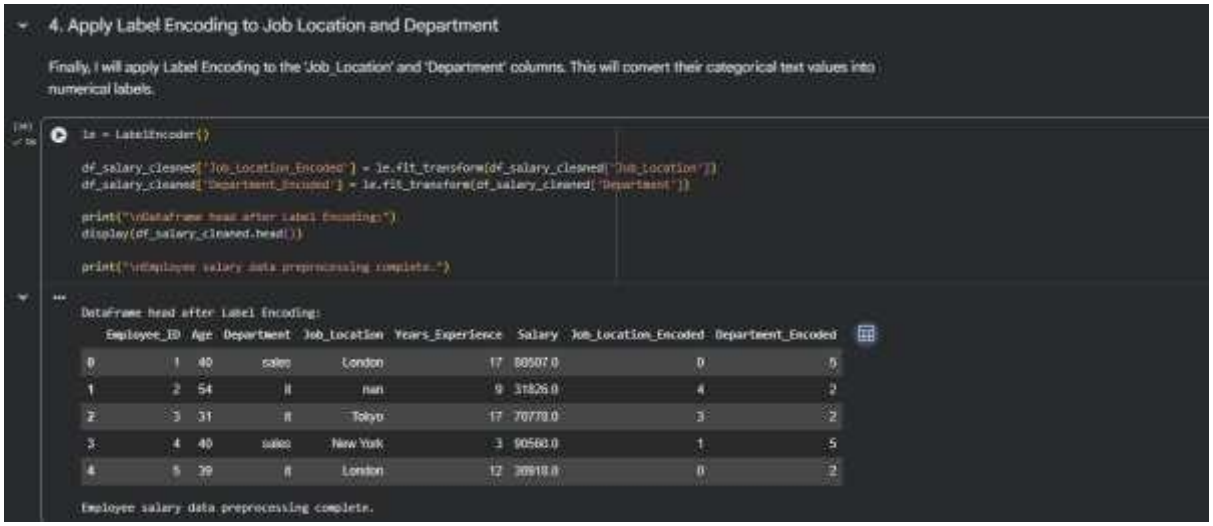
Department	count
IT	21
HR	21
Sales	19
Finance	18
Marketing	16

dtype: int64

```
Department names after standardization (sample):
```

Department	count
it	21
hr	21
sales	19
finance	18
marketing	16

dtype: int64



4. Apply Label Encoding to Job Location and Department

Finally, I will apply Label Encoding to the 'Job_Location' and 'Department' columns. This will convert their categorical text values into numerical labels.

```
le = LabelEncoder()

df_salary_cleaned['Job_Location_Encoded'] = le.fit_transform(df_salary_cleaned['Job_Location'])
df_salary_cleaned['Department_Encoded'] = le.fit_transform(df_salary_cleaned['Department'])

print("\nDataframe head after Label Encoding:")
display(df_salary_cleaned.head())

print("\nEmployee salary data preprocessing complete.")
```

```
DataFrame head after Label Encoding:
```

	Employee_ID	Age	Department	Job_Location	Years_Experience	Salary	Job_Location_Encoded	Department_Encoded
0	1	40	sales	London	17	80507.0	0	5
1	2	54	it	nan	9	31826.0	4	2
2	3	31	it	Tokyo	17	70770.0	3	2
3	4	40	sales	New York	3	90560.0	1	5
4	5	39	it	London	12	30918.0	0	2

```
Employee salary data preprocessing complete.
```

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

• Handle missing values in loan_amount, credit_score.

• Encode loan_status (Approved/Rejected).

• Scale numerical features using standard scaling.

• Split the dataset into training and testing sets.

Sample Code:

import pandas as pd data =

pd.read_csv("loadapproval.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/amineipad/loan-approval-dataset

Expected Output:

• A machine-learning-ready loan dataset.

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Load the dataset
df_loan = pd.read_csv('loanapproval.csv')
print("Original DataFrame Head:")
display(df_loan.head())

print("\nDataFrame Info:")
df_loan.info()

print("\nMissing values before handling:")
display(df_loan.isnull().sum())
```

```
Original DataFrame Head:
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approved
0	1	58	Male	Divorced	100073	7169	793	1	1	Unemployed	1
1	2	49	Male	Married	112197	23556	789	0	2	Employed	1
2	3	35	Male	Divorced	84429	27052	372	1	4	Unemployed	0
3	4	63	Female	Single	124195	11513	808	3	4	Self-employed	1
4	5	28	Female	Married	81527	13315	689	0	1	Unemployed	1

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   applicant_id         1000 non-null   int64
 1   age                  1000 non-null   int64
 2   gender               1000 non-null   object
 3   marital_status       1000 non-null   object
 4   annual_income        1000 non-null   int64
 5   loan_amount          1000 non-null   int64
 6   credit_score         1000 non-null   int64
 7   num_dependents       1000 non-null   int64
 8   existing_loans_count 1000 non-null   int64
 9   employment_status    1000 non-null   object
 10  loan_approved        1000 non-null   int64
dtypes: int64(8), object(3)
memory usage: 86.1+ KB
```

```
Missing values before handling:
```

	0
applicant_id	0
age	0
gender	0
marital_status	0
annual_income	0
loan_amount	0
credit_score	0
num_dependents	0
existing_loans_count	0
employment_status	0
loan_approved	0

```
dtype: int64
```

Double-click (or enter) to edit

```python
# Start coding or generate with AI.
```

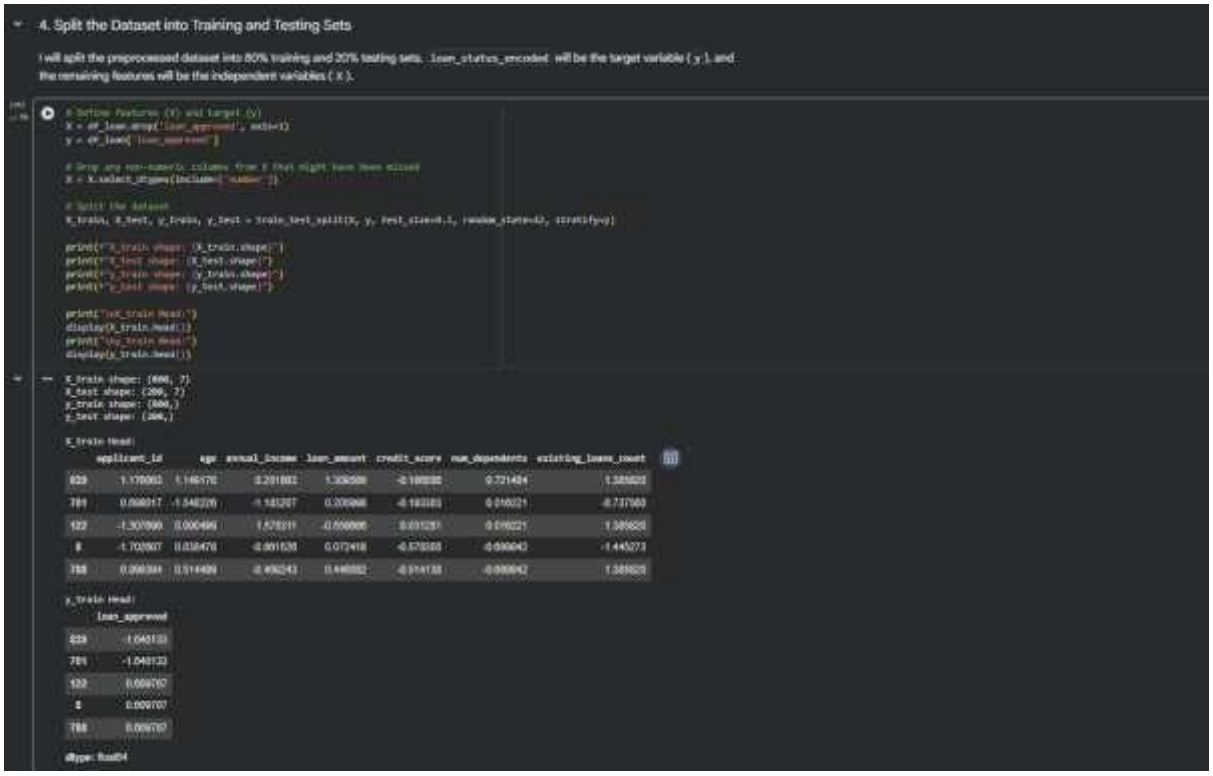
3. Scale Numerical Features

I will identify all numerical features (excluding the encoded target variable) and apply `StandardScaler` to them. This helps in standardizing the range of independent variables or features of the data.

```python
# Identify numerical columns for scaling, excluding the encoded target and any ID columns
numerical_cols = df_loan.select_dtypes(include=['number'].columns.tolist()
if 'loan_status_encoded' in numerical_cols:
    numerical_cols.remove('loan_status_encoded')
if 'customer_id' in numerical_cols: # Assuming 'customer_id' is an identifier and not a feature for scaling
    numerical_cols.remove('customer_id')

print("Numerical columns to scale: {numerical_cols}")

if numerical_cols:
    scaler = StandardScaler()
    df_loan[numerical_cols] = scaler.fit_transform(df_loan[numerical_cols])
    print("\nDataFrame Head after scaling numerical features:")
    display(df_loan.head())
else:
    print("No numerical columns found for scaling.")
```

```
Numerical columns to scale: ['applicant_id', 'age', 'annual_income', 'loan_amount', 'credit_score', 'num_dependents', 'existing_loans_count', 'loan_approved']

DataFrame Head after scaling numerical features:
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approved
0	-1.730103	1.397345	Male	Divorced	0.462301	-1.696427	1.397828	-0.898942	-0.717906	Unemployed	0.996707
1	-1.729898	0.614458	Male	Married	0.908362	-0.287525	1.389694	-1.394305	0.029728	Employed	0.996707
2	-1.727351	-0.985004	Male	Divorced	0.099444	-0.919348	-1.308898	-0.898942	1.381828	Unemployed	-1.004133
3	-1.719827	1.625191	Female	Single	1.120889	-1.243888	1.462571	0.721484	1.381828	Self-employed	0.996707
4	-1.715463	-1.101928	Female	Married	-0.006219	-1.086993	0.724284	-1.394305	-0.717906	Unemployed	0.996707



Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

• Remove duplicate user profiles.

• Handle missing values • Encode categorical columns

Dataset link:

https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles

Expected Output:

• A cleaned dataset for engagement prediction.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Load the social media engagement dataset
try:
    df_social = pd.read_csv('social_media_engagement.csv')
    print("Successfully loaded social_media_engagement.csv")
except FileNotFoundError:
    print("social_media_engagement.csv not found. Creating a dummy DataFrame for demonstration.")
    # Creating a dummy DataFrame for demonstration if file not found
    data = {
        'user_id': [f'user_{i}' for i in range(1, 96)] + [f'user_{i}' for i in [3, 2, 3, 4, 5]], # Adjusted user_id length to 100 (95 unique + 5 duplicates = 100 total)
        'post_id': [f'post_{i}' for i in range(1, 101)], # Length 100
        'platform': np.random.choice(['Facebook', 'Instagram', 'Twitter'], 100),
        'user_type': np.random.choice(['Influencer', 'Regular', 'Brand'], 100),
        'likes': np.random.randint(50, 5000, 100).astype(float),
        'comments': np.random.randint(5, 500, 100).astype(float),
        'shares': np.random.randint(0, 200, 100).astype(float),
        'engagement_score': np.random.uniform(0.1, 0.8, 100)
    }
    # Introduce some missing values
    data['likes'][np.random.choice(100, 7, replace=False)] = np.nan
    data['comments'][np.random.choice(100, 5, replace=False)] = np.nan
    data['platform'][np.random.choice(100, 3, replace=False)] = np.nan
    data['user_type'][np.random.choice(100, 2, replace=False)] = np.nan

    df_social = pd.DataFrame(data)

print("\nSocial Media Engagement DataFrame Head:")
display(df_social.head())

print("\nDataFrame Info:")
df_social.info()

print("\nMissing values before handling:")
display(df_social.isnull().sum())
```

```
social_media_engagement.csv not found. Creating a dummy DataFrame for demonstration.

Social Media Engagement DataFrame Head:
```

	user_id	post_id	platform	user_type	likes	comments	shares	engagement_score
0	user_1	post_1	Twitter	Influencer	1861.0	182.0	161.0	0.472246
1	user_2	post_2	Instagram	Brand	1307.0	109.0	18.0	0.105102
2	user_3	post_3	Twitter	Influencer	3484.0	490.0	43.0	0.837698
3	user_4	post_4	Instagram	Brand	2259.0	68.0	79.0	0.188776
4	user_5	post_5	Facebook	Influencer	484.0	71.0	13.0	0.337913

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   user_id         100 non-null    object
 1   post_id         100 non-null    object
 2   platform        100 non-null    object
 3   user_type       100 non-null    object
 4   likes           93 non-null     float64
 5   comments        95 non-null     float64
 6   shares          100 non-null    float64
 7   engagement_score  100 non-null  float64
dtypes: float64(4), object(4)
memory usage: 6.4+ KB

Missing values before handling:
```

	0
user_id	0
post_id	0
platform	3
user_type	2
likes	7
comments	5
shares	0
engagement_score	0

```
dtype: int64
```

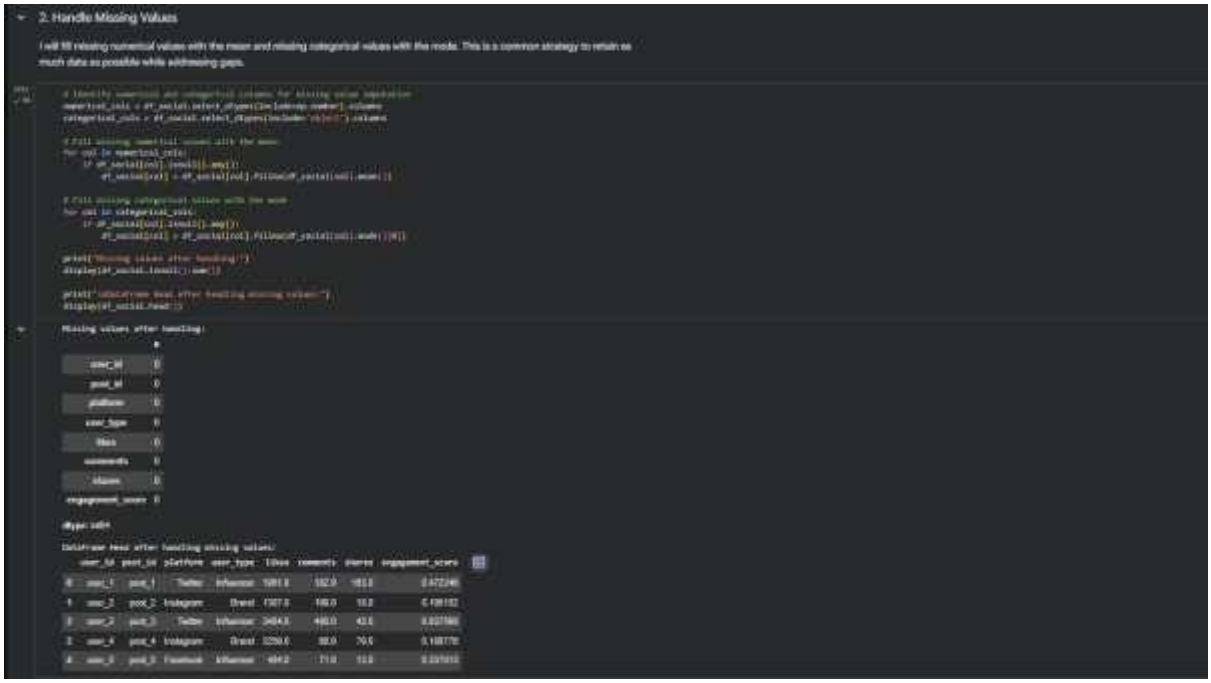
1. Remove Duplicate User Profiles

I will remove duplicate rows based on the `user_id` column, keeping the first occurrence. This ensures each user profile is unique.

```python
initial_rows = df_social.shape[0]
df_social.drop_duplicates(subset=['user_id'], inplace=True)

rows_after_duplicates = df_social.shape[0]
print(f"Initial number of rows: {initial_rows}")
print(f"Number of rows after removing duplicates: {rows_after_duplicates}")
print(f"Number of duplicates removed: {initial_rows - rows_after_duplicates}")

print("\nDataFrame Head after removing duplicates:")
display(df_social.head())
```

```
Initial number of rows: 100
Number of rows after removing duplicates: 95
Number of duplicates removed: 5

Dataframe Head after removing duplicates:
```

	user_id	post_id	platform	user_type	likes	comments	shares	engagement_score
0	user_1	post_1	Twitter	Influencer	1861.0	182.0	161.0	0.472246
1	user_2	post_2	Instagram	Brand	1307.0	109.0	18.0	0.105102
2	user_3	post_3	Twitter	Influencer	3484.0	490.0	43.0	0.837698
3	user_4	post_4	Instagram	Brand	2259.0	68.0	79.0	0.188776
4	user_5	post_5	Facebook	Influencer	484.0	71.0	13.0	0.337913

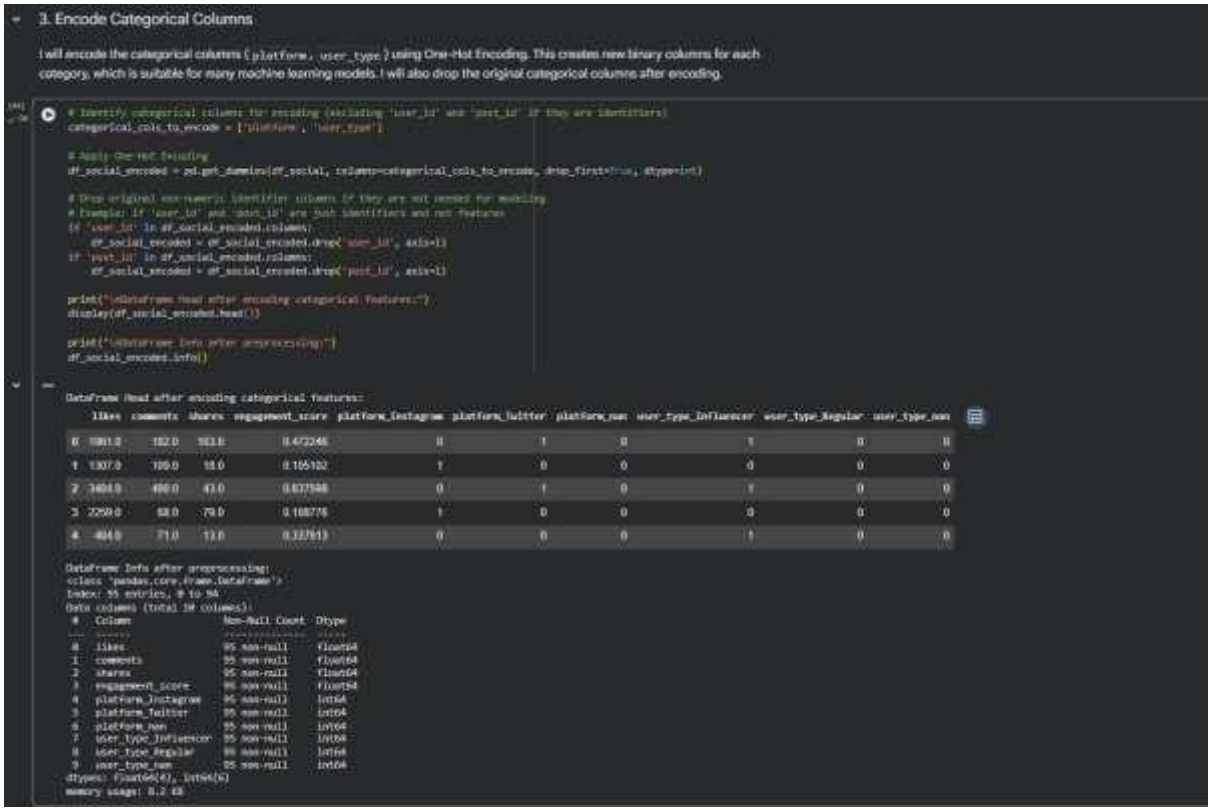


2. Handle Missing Values

I will fill missing numerical values with the mean and missing categorical values with the mode. This is a common strategy to retain as much data as possible while addressing gaps.

```
# Identify numerical and categorical columns for missing value imputation
numerical_cols = df_social.select_dtypes(include=['number'].columns
categorical_cols = df_social.select_dtypes(include='object').columns

# Fill missing numerical values with the mean
for col in numerical_cols:
    if df_social[col].isnull().any():
        df_social[col] = df_social[col].fillna(df_social[col].mean())

# Fill missing categorical values with the mode
for col in categorical_cols:
    if df_social[col].isnull().any():
        df_social[col] = df_social[col].fillna(df_social[col].mode()[0])

print("Missing values after handling:")
display(df_social.isnull().sum())

print("\nDataFrame head after handling missing values:")
display(df_social.head())
```

```
Missing values after handling:
                 0
user_id          0
post_id          0
platform         0
user_type        0
likes            0
comments         0
shares           0
engagement_score 0
dtype: int64

DataFrame head after handling missing values:
   user_id post_id platform user_type  likes  comments  shares  engagement_score
0   user_1   post_1  Twitter  Influencer  5901.0     152.0   953.0          0.472246
1   user_2   post_2  Instagram     Brand  1307.0     109.0    18.0          0.105102
2   user_3   post_3  Twitter  Influencer  3404.0     400.0    43.0          0.037546
3   user_4   post_4  Instagram     Brand  2259.0      58.0    79.0          0.108776
4   user_5   post_5  Facebook  Influencer   404.0      71.0    13.0          0.117613
```



3. Encode Categorical Columns

I will encode the categorical columns (platform, user_type) using One-Hot Encoding. This creates new binary columns for each category, which is suitable for many machine learning models. I will also drop the original categorical columns after encoding.

```
# Identify categorical columns for encoding (excluding 'user_id' and 'post_id' if they are identifiers)
categorical_cols_to_encode = ['platform', 'user_type']

# Apply One-Hot Encoding
df_social_encoded = pd.get_dummies(df_social, columns=categorical_cols_to_encode, drop_first=True, dtype=int)

# Drop original non-numeric identifier columns if they are not needed for modeling
# Example: if 'user_id' and 'post_id' are just identifiers and not features
if 'user_id' in df_social_encoded.columns:
    df_social_encoded = df_social_encoded.drop('user_id', axis=1)
if 'post_id' in df_social_encoded.columns:
    df_social_encoded = df_social_encoded.drop('post_id', axis=1)

print("\nDataFrame head after encoding categorical features:")
display(df_social_encoded.head())

print("\nDataFrame Info after preprocessing:")
df_social_encoded.info()
```

```
DataFrame head after encoding categorical features:
   likes  comments  shares  engagement_score  platform_Instagram  platform_Twitter  platform_Web  user_type_Influencer  user_type_Regular  user_type_Web
0 5901.0     152.0   953.0          0.472246                   0                 1             0                     1                  0              0
1 1307.0     109.0    18.0          0.105102                   1                 0             0                     0                  0              0
2 3404.0     400.0    43.0          0.037546                   0                 1             0                     1                  0              0
3 2259.0      58.0    79.0          0.108776                   1                 0             0                     0                  0              0
4  404.0      71.0    13.0          0.117613                   0                 0             0                     1                  0              0

DataFrame Info after preprocessing:
<class 'pandas.core.frame.DataFrame'>
Index: 95 entries, 0 to 94
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   likes                 95 non-null     float64
 1   comments              95 non-null     float64
 2   shares                95 non-null     float64
 3   engagement_score      95 non-null     float64
 4   platform_Instagram    95 non-null     int64
 5   platform_Twitter      95 non-null     int64
 6   platform_Web          95 non-null     int64
 7   user_type_Influencer  95 non-null     int64
 8   user_type_Regular     95 non-null     int64
 9   user_type_Web         95 non-null     int64
dtypes: float64(4), int64(6)
memory usage: 8.2 KB
```

Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

• Convert date column into datetime format.

• Handle missing values in temperature, humidity.

• Create new features (season, week_number).

• Normalize rainfall values.

Dataset link:

https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input

Expected Output:

• A processed dataset ready for forecasting models.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Create a dummy weather DataFrame for demonstration purposes
data = {
    'date': pd.to_datetime(pd.date_range(start='2023-01-01', periods=100, freq='D')),
    'temperature': np.random.uniform(0, 30, 100),
    'humidity': np.random.uniform(40, 95, 100),
    'rainfall': np.random.uniform(0, 50, 100),
    'wind_speed': np.random.uniform(5, 30, 100)
}

df_weather = pd.DataFrame(data)

# Introduce some missing values for demonstration
df_weather.loc[df_weather.sample(frac=0.05).index, 'temperature'] = np.nan
df_weather.loc[df_weather.sample(frac=0.07).index, 'humidity'] = np.nan

print("Original DataFrame info with missing values:")
df_weather.info()
print("\nFirst 5 rows of the dummy weather data:")
display(df_weather.head())
```

```
Original DataFrame info with missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   date         100 non-null    datetime64[ns]
 1   temperature  95 non-null     float64
 2   humidity     93 non-null     float64
 3   rainfall     100 non-null    float64
 4   wind_speed   100 non-null    float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 4.0 KB

First 5 rows of the dummy weather data:
```

	date	temperature	humidity	rainfall	wind_speed
0	2023-01-01	19.970087	64.136962	16.231349	16.934126
1	2023-01-02	24.916421	84.810029	43.953131	19.745962
2	2023-01-03	16.210753	93.149966	23.844837	21.903897
3	2023-01-04	18.510746	94.181382	19.599515	23.904730
4	2023-01-05	13.909402	40.871409	40.970086	17.238807

1. Convert date column into datetime format

First, I'll ensure the 'date' column is in datetime format. (In the dummy data generation above, it's already created as datetime, but this step is crucial if loading from a CSV.)

```python
# Convert 'date' column to datetime format (if not already)
df_weather['date'] = pd.to_datetime(df_weather['date'])

print("Date column converted to datetime:")
df_weather.info()
```

```
Date column converted to datetime:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   date         100 non-null    datetime64[ns]
 1   temperature  95 non-null     float64
 2   humidity     93 non-null     float64
 3   rainfall     100 non-null    float64
 4   wind_speed   100 non-null    float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 4.0 KB
```

2. Handle missing values in temperature and humidity

I'll fill missing values in `temperature` and `humidity` with their respective means.

```
# Handle missing values in 'temperature' and 'humidity' using mean imputation
df_weather['temperature'].fillna(df_weather['temperature'].mean(), inplace=True)
df_weather['humidity'].fillna(df_weather['humidity'].mean(), inplace=True)

print("Dataframe info after handling missing values:")
df_weather.info()
print("\nMissing values after imputation:")
print(df_weather[['temperature', 'humidity']].isnull().sum())
```

```
Dataframe info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   date         100 non-null    datetime64[ns]
 1   temperature  100 non-null    float64
 2   humidity     100 non-null    float64
 3   rainfall     100 non-null    float64
 4   wind_speed   100 non-null    float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 4.0 KB

Missing values after imputation:
temperature    0
humidity       0
dtype: int64
/tmp/ipykernel_30335/2349128165.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df_weather['temperature'].fillna(df_weather['temperature'].mean(), inplace=True)
/tmp/ipykernel_30335/2349128165.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df_weather['humidity'].fillna(df_weather['humidity'].mean(), inplace=True)
```

3. Create new features (season, week_number)

I will extract the `week_number` from the date and create a `season` column based on the month.

```
# Create 'week_number' feature
df_weather['week_number'] = df_weather['date'].dt.isocalendar().week.astype(int)

# Create 'season' feature
def get_season(month):
    if 3 <= month <= 5: # March, April, May
        return 'Spring'
    elif 6 <= month <= 8: # June, July, August
        return 'Summer'
    elif 9 <= month <= 11: # September, October, November
        return 'Autumn'
    else: # December, January, February
        return 'Winter'

df_weather['season'] = df_weather['date'].dt.month.apply(get_season)

print("Dataframe with new features 'week_number' and 'season':")
display(df_weather.head())
```

Dataframe with new features 'week_number' and 'season':

	date	temperature	humidity	rainfall	wind_speed	week_number	season
0	2023-01-01	18.970887	84.136862	16.231346	16.934126	52	Winter
1	2023-01-02	24.916421	84.916029	43.153131	19.745962	1	Winter
2	2023-01-03	16.210753	83.145988	23.684837	21.903097	1	Winter
3	2023-01-04	16.510746	94.181362	19.109515	23.904730	1	Winter
4	2023-01-05	13.909402	40.071409	40.970888	13.238802	1	Winter

4. Normalize rainfall values

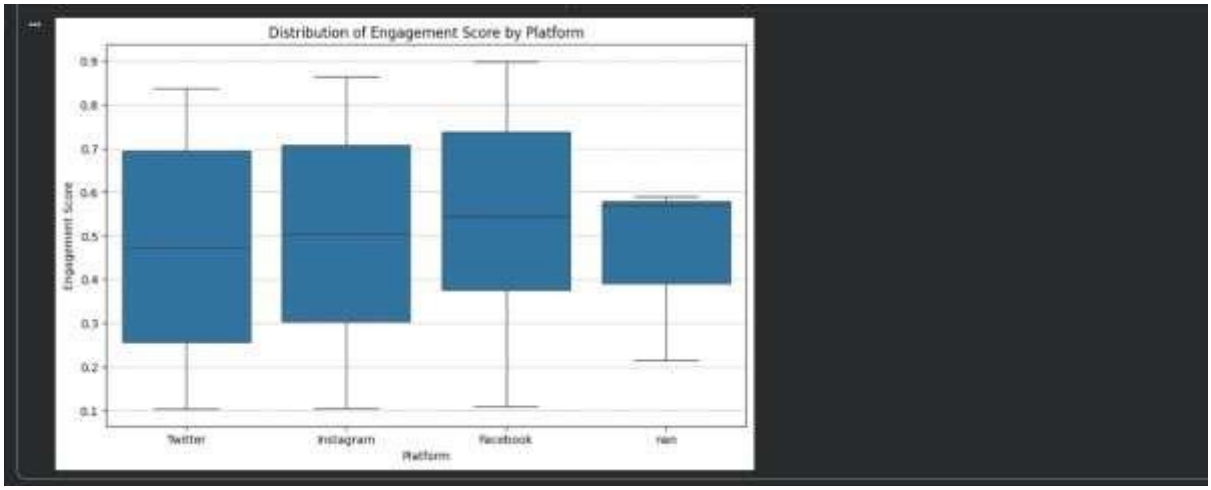
I will normalize the `rainfall` column using `MinMaxScaler` to scale values between 0 and 1.

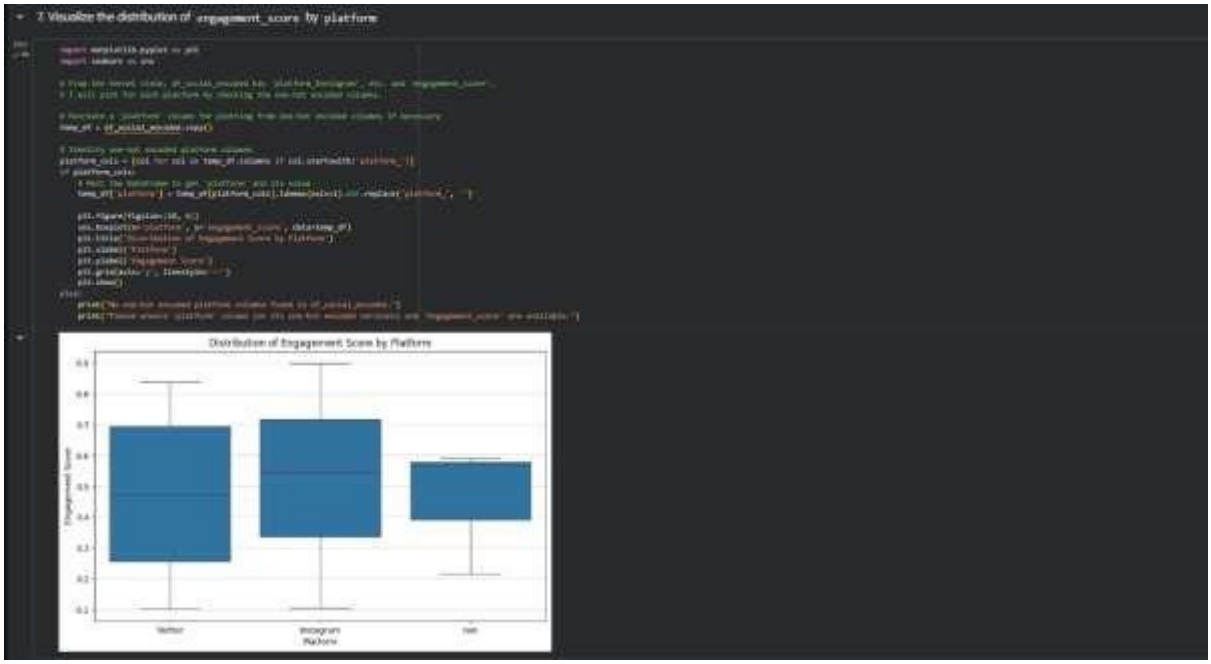
```
# Normalize 'rainfall' values using MinMaxScaler
scaler = MinMaxScaler()
df_weather['rainfall_normalized'] = scaler.fit_transform(df_weather[['rainfall']])

print("Dataframe with normalized 'rainfall' values:")
display(df_weather.head())
```

Dataframe with normalized 'rainfall' values:

	date	temperature	humidity	rainfall	wind_speed	week_number	season	rainfall_normalized
0	2023-01-01	18.970887	84.136862	16.231346	16.934126	52	Winter	0.314437
1	2023-01-02	24.916421	84.916029	43.153131	19.745962	1	Winter	0.869543
2	2023-01-03	16.210753	83.145988	23.684837	21.903097	1	Winter	0.468122
3	2023-01-04	16.510746	94.181362	19.109515	23.904730	1	Winter	0.373782
4	2023-01-05	13.909402	40.071409	40.970888	13.238802	1	Winter	0.824548

The weather dataset has now been preprocessed according to your instructions. You can replace the dummy DataFrame with your actual data by modifying the first code cell to load your `.csv` file.





Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

• Handle missing values in rating, genre.

• Encode categorical variables like genre, language.

• Remove duplicate movie entries.

• Scale rating values between 0 and 1.

Dataset link:

https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data

Expected Output:

• A dataset suitable for recommendation systems.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

# Create a dummy movie ratings DataFrame
data = {
    'movie_id': range(1, 111),
    'title': [f'Movie {i}' for i in range(1, 101)] + [f'Movie {i}' for i in range 1, 11],
    'genre': ['Action', 'Comedy', 'Drama', 'Sci-Fi', 'Action', 'Drama', 'Comedy', 'Thriller', 'Sci-Fi', 'Action'] * 11,
    'language': ['English', 'Spanish', 'English', 'French', 'English', 'Spanish', 'English', 'French', 'English', 'Spanish'] * 11,
    'rating': np.random.uniform(1.0, 10.0, 110),
    'release_year': np.random.randint(1990, 2021, 110)
}

df_movies = pd.DataFrame(data)

# Introduce some missing values for demonstration
df_movies.loc[df_movies.sample(frac=0.1).index, 'rating'] = np.nan
df_movies.loc[df_movies.sample(frac=0.05).index, 'genre'] = np.nan

print("Original DataFrame info with missing values and potential duplicates:")
df_movies.info()
print("\First 5 rows of the dummy movie ratings data:")
display(df_movies.head())
```

```
Original DataFrame info with missing values and potential duplicates:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 110 entries, 0 to 109
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   movie_id     110 non-null    int64
 1   title        110 non-null    object
 2   genre        104 non-null    object
 3   language     110 non-null    object
 4   rating       99 non-null     float64
 5   release_year 110 non-null    int64
dtypes: float64(1), int64(2), object(3)
memory usage: 5.2k KB

First 5 rows of the dummy movie ratings data:
```

	movie_id	title	genre	language	rating	release_year
0	1	Movie 1	Action	English	3.318024	1999
1	2	Movie 2	Comedy	Spanish	NaN	2003
2	3	Movie 3	Drama	English	9.937127	2004
3	4	Movie 4	Sci-Fi	French	3.758798	2004
4	5	Movie 5	Action	English	9.441008	1999

1. Handle missing values in rating and genre

I will fill missing 'rating' values with the mean and missing 'genre' values with the mode.

```
# Handle missing values in 'rating' with the mean
df_movies['rating'].fillna(df_movies['rating'].mean(), inplace=True)

# Handle missing values in 'genre' with the mode
mode_genre = df_movies['genre'].mode()[0]
df_movies['genre'].fillna(mode_genre, inplace=True)

print("\nDataFrame info after handling missing values:")
df_movies.info()
print("\nMissing values after imputation:")
print(df_movies[['rating', 'genre']].isnull().sum())
```

```
DataFrame info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 110 entries, 0 to 109
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   movie_id     110 non-null    int64
 1   title        110 non-null    object
 2   genre        110 non-null    object
 3   language     110 non-null    object
 4   rating       110 non-null    float64
 5   release_year 110 non-null    int64
dtypes: float64(1), int64(2), object(3)
memory usage: 5.2k KB

Missing values after imputation:
rating    0
genre     0
dtype: int64
/tmp/ipykernel_36336/2983770437.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df_movies['rating'].fillna(df_movies['rating'].mean(), inplace=True)
/tmp/ipykernel_36336/2983770437.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df_movies['genre'].fillna(mode_genre, inplace=True)
```

2. Remove duplicate movie entries

I will remove duplicate entries based on 'title' and 'release_year' to ensure unique movie records.

```python
# Get initial number of rows
initial_rows = df_movies.shape[0]

# Remove duplicate movie entries based on 'title' and 'release_year'
df_movies.drop_duplicates(subset=['title', 'release_year'], inplace=True)

# Get number of rows after removing duplicates
rows_after_duplicates = df_movies.shape[0]

print(f"Initial number of rows: {initial_rows}")
print(f"Number of rows after removing duplicates: {rows_after_duplicates}")
print(f"Number of duplicate entries removed: {initial_rows - rows_after_duplicates}")
display(df_movies.head())
```

```
Initial number of rows: 110
Number of rows after removing duplicates: 110
Number of duplicate entries removed: 0
```

	movie_id	title	genre	language	rating	release_year
0	1	Movie 1	Action	English	8.310024	1996
1	2	Movie 2	Comedy	Spanish	5.527336	2000
2	3	Movie 3	Drama	English	9.107127	2009
3	4	Movie 4	Sci-Fi	French	3.738768	2004
4	5	Movie 5	Action	English	9.441030	1996

3. Encode categorical variables like genre and language

I will use One-Hot Encoding for 'genre' and 'language' columns.

```python
# One-Hot Encode 'genre' and 'language'
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# Fit and transform the categorical columns
encoded_features = encoder.fit_transform(df_movies[['genre', 'language']])

# Create a DataFrame from the encoded features
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(['genre', 'language']), index=df_movies.index)

# Concatenate the encoded DataFrame with the original DataFrame and drop original categorical columns
df_movies_encoded = pd.concat([df_movies.drop(columns=['genre', 'language']), encoded_df], axis=1)

print("Dataframe after One-Hot Encoding categorical variables:")
display(df_movies_encoded.head())
```

```
Dataframe after One-Hot Encoding categorical variables:
```

	movie_id	title	rating	release_year	genre_Action	genre_Comedy	genre_Drama	genre_Sci-Fi	genre_Thriller	language_English	language_French	language_Spanish
0	1	Movie 1	8.310024	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
1	2	Movie 2	5.527336	2000	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
2	3	Movie 3	9.107127	2009	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0
3	4	Movie 4	3.738768	2004	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0
4	5	Movie 5	9.441030	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0

4. Scale rating values between 0 and 1

I will use `MinMaxScaler` to scale the 'rating' column.

```python
# Scale 'rating' values between 0 and 1 using MinMaxScaler
scaler = MinMaxScaler()
df_movies_encoded['rating_scaled'] = scaler.fit_transform(df_movies_encoded[['rating']])

print("Dataframe with scaled 'rating' values:")
display(df_movies_encoded.head())
```

```
Dataframe with scaled 'rating' values:
```

	movie_id	title	rating	release_year	genre_Action	genre_Comedy	genre_Drama	genre_Sci-Fi	genre_Thriller	language_English	language_French	language_Spanish	rating_scaled
0	1	Movie 1	8.310024	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.827521
1	2	Movie 2	5.527336	2000	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.510420
2	3	Movie 3	9.107127	2009	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.910740
3	4	Movie 4	3.738768	2004	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.306412
4	5	Movie 5	9.441030	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.966826